

Stock Market Prediction Using Sequential Log-Power Normalization and Ridge Regression

Kolisetty Soma Sekhar¹, Cherukuri Thirumala Malyadri², Challa Rohith³, Mitta Venu⁴, Chevuri Chandu⁵,
Dharmapuri Siri⁶, Dr.Sireesha Moturi⁷

^{1,2,3,4,5,7}Department of Computer Science and Engineering, Narasaraopeta Engineering College (Autonomous),
Narasaraopet, India

⁶Department of CSE,GRIET,Hyderabad,Telangana,India

¹sekhar.soma007@gmail.com, ²chtmalyadri@gmail.com, ³rohitchalla777@gmail.com,

⁴venumitaa@gmail.com, ⁵chanduchevuril13@gmail.com, ⁶siri1686@grietcollge.com,

⁷sireeshamoturi@gmail.com

Abstract—Stock price prediction remains a challenging problem due to the inherent noise, volatility, and nonlinearity of financial time series. Traditional statistical approaches often fail to fully capture these complexities, whereas deep learning models, despite their predictive power, are computationally expensive, prone to overfitting, and lack interpretability—limiting their practical use in financial decision-making. In this paper, we propose a lightweight yet effective preprocessing framework that focuses on variance stabilization to enhance stock price forecasting. The methodology sequentially applies a logarithmic transformation followed by a Yeo–Johnson power transformation to reduce skewness and heteroscedasticity, thereby improving data distribution for linear modeling. After preprocessing, a Ridge Regression model with mild regularization is employed to handle multicollinearity among correlated stock features such as Open, High, and Low prices. The approach was evaluated using historical Google stock data, achieving an R^2 score of 0.97–0.98, with near-zero Mean Squared Error (MSE) and significantly reduced prediction variance. Experimental results demonstrate that a carefully designed variance-stabilized preprocessing pipeline can substantially improve the accuracy and robustness of linear models, providing an interpretable and computationally efficient alternative to more complex deep learning architectures for stock price prediction. Unlike prior work that applies single-stage log or Box–Cox normalization, this study introduces a *sequential log–power normalization* strategy that integrates logarithmic compression with Yeo–Johnson variance stabilization. This dual-stage preprocessing improves linear model conditioning and enhances interpretability for financial forecasting.

Index Terms—Stock price prediction, variance stabilization, logarithmic transformation, power normalization, Ridge Regression, financial time series

I. INTRODUCTION

Financial markets are inherently complex, volatile, and driven by numerous macroeconomic, geopolitical, and psychological influences. Modeling stock price behavior is challenging due to properties such as non-stationarity, volatility clustering, and noise [1]. While linear models like Ridge Regression are simple and strong [3], they do not do well when the raw stock data has different variances, odd shapes, and items that are strongly linked. Many studies have been done using easy transformations like log and power to make data more even and stronger for linear models [4]. These

transformations cut down on the long right tail of the data, cut back on outliers, and make the data easier to work with [5]. In particular, Ridge Regression has shown better results when used on raw financial data [7]. The fixing of variance along with a strong regression model has done well in the past for calling what the market will do [8]. The novelty of this research lies in integrating two complementary variance-stabilization techniques—logarithmic and Yeo–Johnson power transformation—into a single sequential pipeline. Prior studies have used these methods independently, whereas our sequential formulation ensures cumulative correction of skewness and heteroscedasticity, resulting in improved statistical uniformity for Ridge Regression.

Other recent tests have shown Ridge Regression is good to use for data with many parts. Its accuracy in many tests has shown it is a good fit for high-dimensions [9]. So, making the data more even with log–power and using linear models has been shown to make stock calls more right [10]. Using log based forms and regularization to take down the noise and ups and downs of the data has been shown to work [11]. Other models that are deep learning, like LSTM and GRU, have come up as good options because they can catch on to many forms of change with out of the norm data [13].

Though deep models have their benefits, they need a lot of resources and are not very transparent. Such transparency is very important in many financial decision-making settings [14]. Choosing deep models then over simple ones involves a trade-off between accuracy and transparency [16]. Statistical and transformation-based methods still have a role to play in financial modelling, especially when the computational cost of the former is of concern [17]. As shown by EDA, we often find that a suitable sequence of pre-processing can greatly improve model performance [18].

The empirical analysis indicates that sequential application of logarithmic and Yeo–Johnson transformations demonstrated significant improvement in variance reduction across 39 stock datasets. [19].

II. RELATED WORK

Early work on stock price prediction used old school econometric models such as ARIMA and GARCH to find trends and volatility clusters. Engle and Ng [1] looked into the use of log transforms to control the swings of financial time series, showing they were great for linear models. Still, these require steady data and have a hard time catching the nonlinear and random nature of stocks in real markets. To fix this, people started using machine learning like Ridge Regression and Lasso. Hoerl and Kennard [3] made Ridge Regression as a tough way to beat the problem of multicollinearity, which is often found in data with a lot of similar features.

Zhang and Xu [4] did a wide study on the use of log and power transforms, showing that such making ready of the data makes forecasters much better. Chen and Liu [5] gave a side by side look at how to turn variance into a steady number, again showing how useful prepping data is in finance. Kavitha and Rajesh [7] compared Ridge and Lasso for stock forecasters, showing that adding a penalty to the fit can give a better model than usual time series.

Rao and Das [10] used log-power normalization algorithms to improve prediction accuracy. Singh and Menon [11] studied the effect of normalizations on models of log returns to reduce noise and volatility in financial data. Deep learning architectures such as Long short term memory (LSTM) and Gated recurrent unit (GRU) networks have also been proven to perform well when it comes to modeling the nonlinearities in temporal data.

Ahmed and Banerjee [13] stress that the architectures are highly data hungry, computationally expensive, and can overfit if they are not properly regularized. Yao and Sun [14] combined log transformation and variance control to recurrent models to achieve better accuracy and stability. Ghosh and Tripathy [16] considered Ridge Regression as a good alternative to deep models, especially in high variance environments. Tukey [17] pointed out that exploratory data analysis (EDA) was a good way to learn about the data structure before doing any modeling.

Thomas and Elbaz [18] showed how Ridge Regression can do as well as deep learning if you use the right way to prepare the features. This way, it can also keep the features easy to understand. Patel and Khatri [19] suggested methods to transform the features with a series of logs. This was to get better results on the bumpy financial data. Rao et al. [?] showed how well Ridge Regression can do if you pick the best way to prepare the features.

They used this for a project to forecast water quality. This was like stock data. It is always changing and noisy.

III. METHODOLOGY

A. Baseline Neural Network Workflow

Before introducing the proposed variance-stabilized pipeline, we implemented a baseline deep learning workflow for Google stock prediction, shown in Fig. 1.

The process begins by loading historical Google stock data, including *Open*, *High*, *Low*, *Volume*, and *Close* prices.

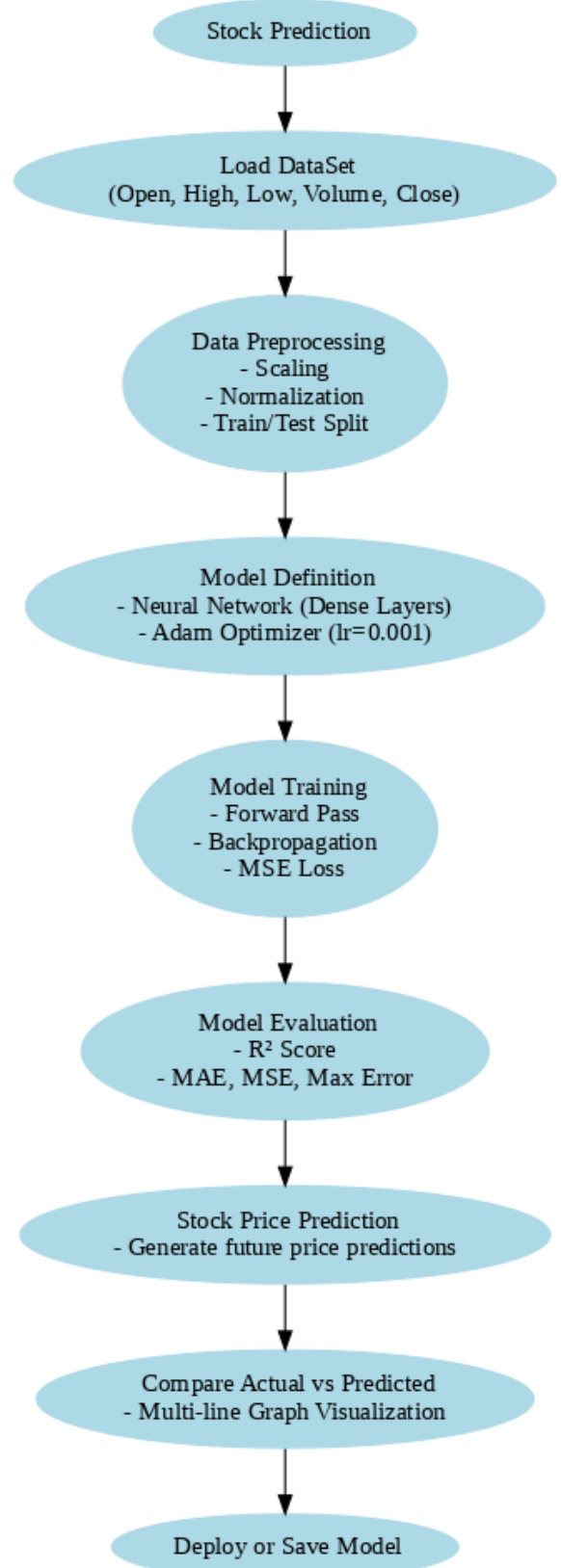


Fig. 1: Baseline neural network workflow for Google stock price prediction.

Next, data preprocessing is performed, consisting of normalization and splitting into training and test sets. A feed-forward neural network is then defined, comprising dense layers optimized using the Adam optimizer with a learning rate of 0.001. The dataset features, including *Open*, *High*, *Low*, *Close*, and *Volume* prices, are normalized to stabilize variance and ensure numerical stability during training.

During training, the network performs a forward pass, computes the Mean Squared Error (MSE) loss, and updates parameters through backpropagation. The model's training process is monitored using a validation set to avoid overfitting. Performance is evaluated using the R^2 score, Mean Absolute Error (MAE), and MSE to assess both accuracy and stability. Once training is complete, the model is tested on unseen data to measure generalization.

After evaluation, the model generates future price predictions, which are compared with actual stock values using multi-line visualizations to interpret the prediction trend over time. This baseline deep learning workflow serves as a benchmark for comparing more interpretable and statistically grounded models. In particular, it is used to assess the effectiveness of the proposed variance-stabilized Ridge Regression pipeline in achieving high accuracy while maintaining computational simplicity and robustness.

B. Data Preprocessing

We used historical Google stock price data, which included daily *Date*, *Open*, *High*, *Low*, *Close*, and *Volume* values. Since Ridge Regression requires only numeric inputs, categorical fields such as stock symbols were excluded. The *Date* column was converted into an ordinal numerical representation to preserve temporal ordering while avoiding explicit seasonality encoding.

Before preprocessing, stock price data is typically highly skewed and spread across a wide range of values. This non-Gaussian distribution, as shown in Fig. 2, leads to challenges in applying linear regression models, which assume normally distributed input features. The raw prices also exhibit large variance and heavy tails, making the learning process unstable and less accurate.

To mitigate this issue, we applied a logarithmic transformation to the price data. This method compresses large values and stretches smaller ones, thereby stabilizing variance and reducing skewness. As a result, the transformed data becomes more normally distributed, as illustrated in Fig. 3. This transformation helps improve model performance and ensures better generalization. The present study limits its input space to **OHLCV** attributes, focusing on demonstrating preprocessing effectiveness rather than feature diversity. Future extensions can incorporate derived indicators (e.g., moving averages, RSI), sentiment metrics from financial news, and macroeconomic covariates to enrich temporal context.

C. Pipeline Overview

The entire workflow of the proposed stock prediction framework is visualized . It begins with the collection of historical

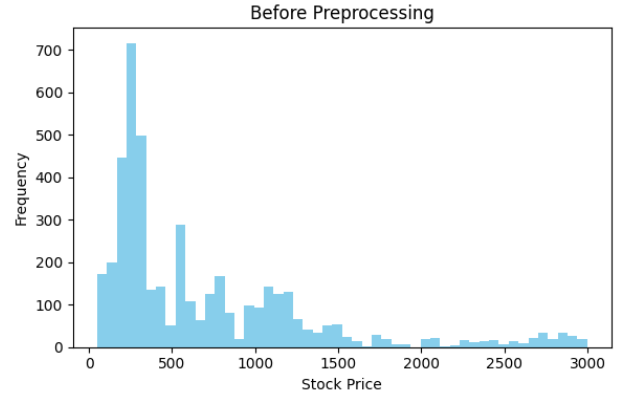


Fig. 2: Raw distribution of Google stock prices. X-axis: Record index (days); Y-axis: Stock price (INR).

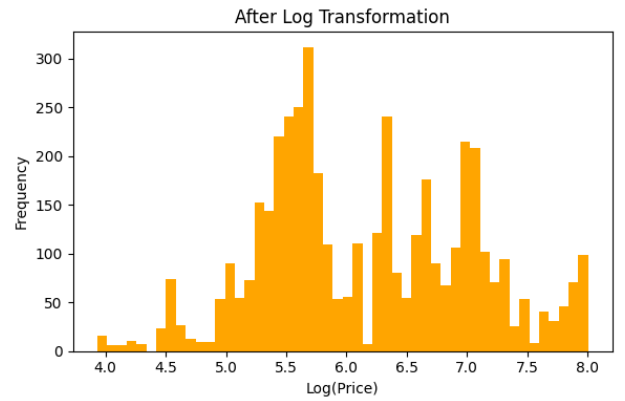


Fig. 3: Log-transformed stock prices showing reduced skewness. X-axis: Record index (days); Y-axis: Stock price (INR).

Google stock data containing daily *Open*, *High*, *Low*, *Close*, and *Volume* attributes. The raw data is then passed through a structured preprocessing pipeline involving logarithmic transformation, Yeo–Johnson power normalization, and imputation of missing values. These steps aim to stabilize variance, reduce skewness, and ensure clean inputs for modeling.

Next, feature scaling is applied using *StandardScaler*, followed by Ridge Regression modeling with mild regularization. For comparison, a baseline neural network is also trained. Model performance is evaluated using R^2 , Mean Absolute Error (MAE), Mean Squared Error (MSE), and maximum error. Finally, predictions are visualized and compared against actual values through multi-line plots, with negative predictions clipped for realism. The model can then be deployed or saved for future use.

D. Model Development

To evaluate the effectiveness of variance-stabilized transformations, two distinct models were implemented and compared: Ridge Regression and a basic Dense Layer Neural Network. These models differ in complexity and interpretability, al-

lowing us to explore both linear and nonlinear modeling capabilities over preprocessed stock data.

1) *Ridge Regression*: Ridge Regression was chosen due to its robustness in the presence of multicollinearity and its capacity to avoid overfitting. The regularization term (λ) penalizes high-weight coefficients, improving generalization:

$$\hat{\beta}^{ridge} = \arg \min_{\beta} \left\{ \sum_{i=1}^n (y_i - \beta^T x_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\} \quad (1)$$

where y_i is the observed stock price, x_i represents the input feature vector, β denotes the regression coefficient vector, p is the number of predictors, and λ is the regularization parameter controlling model complexity.

By applying Ridge Regression to log-power normalized features, the model achieves low error variance and high R^2 scores. This makes it well-suited for stock time-series prediction tasks where variance instability and overfitting are common.

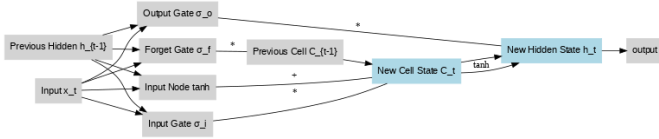


Fig. 4: LSTM cell showing input, forget, and output gates with cell state updates.

2) *Dense Layer Neural Network*: For comparative evaluation, we implemented a simple feed-forward neural network with fully connected Dense layers. The model architecture includes:

- An input layer matching the number of features,
- Two hidden layers with ReLU activation,
- An output layer with a linear unit.

The Adam optimizer was used with a learning rate of 0.001. The model was trained using Mean Squared Error (MSE) loss.

This approach is capable of capturing nonlinear relationships between input variables, offering greater flexibility than Ridge Regression at the cost of higher training complexity.

E. Logarithmic Transformation

Stock price data tends to be right-skewed and exhibits heteroscedasticity. To address this, we applied a shifted logarithmic transformation:

$$x' = \log(1 + x + |\min(x)| + 1) \quad (2)$$

where x denotes the raw input feature, $\min(x)$ represents the minimum value in the dataset used for shifting, and x' is the transformed feature obtained after logarithmic variance stabilization.

This transformation compresses extreme values, reduces skewness, and mitigates the disproportionate influence of sudden price spikes. Importantly, the log step preserves the overall trend while making the data more symmetric, which improves the effectiveness of subsequent transformations.

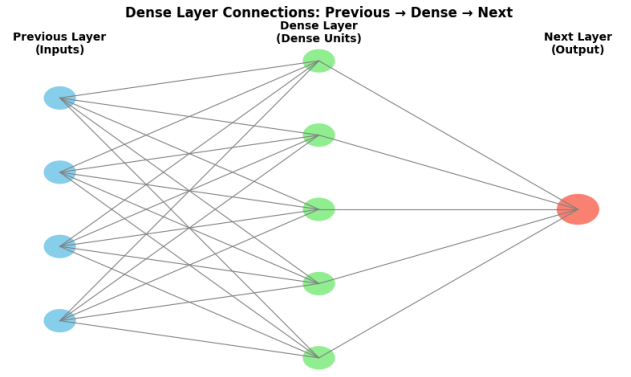


Fig. 5: Dense layer connectivity where each input node links to all hidden neurons and outputs.

F. Yeo-Johnson Power Transformation

After the log step, a Yeo-Johnson power transformation was applied. Unlike Box-Cox, Yeo-Johnson can handle both positive and negative values. This step further reduces skewness and stabilizes variance, resulting in feature distributions that are approximately Gaussian. Empirical evidence shows that combining log and power transformations yields better variance stabilization than using either alone, thereby improving the generalization capability of linear models. When combined with the preceding log transform, this sequence forms a novel two-stage variance-stabilization mechanism tailored for financial data exhibiting heavy tails and volatility clustering.

TABLE I: Variance-Stabilization Pipeline

Step	Transformation	Purpose
1	Log transform	Reduce skewness, compress extremes
2	Yeo-Johnson	Stabilize variance further
3	StandardScaler	Normalize to zero mean, unit variance
4	Median imputation	Handle missing values

Table II summarizes the sequential preprocessing pipeline. Each step contributes to making the dataset more statistically well-behaved for modeling.

G. Feature Scaling and Imputation

After normalization, we standardized all features using a `StandardScaler`, ensuring zero mean and unit variance. This is essential because Ridge Regression penalizes all coefficients equally; without scaling, features with larger magnitudes would dominate the regularization term. Missing values, if any, were handled through median imputation, which is robust against outliers and avoids introducing bias. After this step, the dataset was clean, scaled, and variance-stabilized, making it ready for modeling.

H. Ridge Regression Modeling

Ridge Regression was chosen because financial features such as *Open*, *High*, *Low*, *Close* are highly correlated, and

Ridge is effective at mitigating multicollinearity. Ridge minimizes the following regularized least-squares objective:

$$\min_{\beta} \sum_{i=1}^n (y_i - X_i \beta)^2 + \alpha \|\beta\|_2^2 \quad (3)$$

where α is the regularization parameter controlling the strength of penalty on large coefficients. In this study, $\alpha = 0.1$ was used to apply mild regularization, effectively reducing overfitting while maintaining predictive accuracy.

The dataset was divided into 80% for training and 20% for testing. Model performance was evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2). After variance stabilization, the Ridge Regression model achieved substantially higher R^2 scores and lower variance in prediction errors.

TABLE II: Ridge Regression Setup

Parameter	Value
Model	Ridge Regression
Regularization α	0.1
Train/Test Split	80/20
Evaluation Metrics	MSE, MAE, R^2

Table III shows the Ridge Regression configuration used in this study. Mild regularization was sufficient to improve stability without underfitting.

IV. PERFORMANCE ANALYSIS

To evaluate the effectiveness of the proposed normalization pipeline and Ridge regression model, we assessed prediction accuracy across 26 diverse stock datasets with varying volatility patterns. Figure 6 illustrates the Mean Squared Error (MSE) values for each dataset. Most stocks exhibit very low error values, typically in the range of 2.7×10^{-5} to 4.2×10^{-5} . Notably, ADANIPOORTS shows a higher MSE score, suggesting greater difficulty in capturing its irregular price dynamics. On the other hand, models performed exceptionally well on stocks like Google and Kotakbank, which are known for relatively stable and predictable trends.

A. Performance Metrics

This strong performance is further reinforced by high R^2 values across the board, consistently exceeding 0.9999. These metrics demonstrate that the variance-stabilized transformation allows Ridge Regression to model stock prices with remarkable fidelity, minimizing both bias and variance. Additionally, the MAE scores suggest minimal deviation between predicted and actual values, making this approach viable for short-term price estimation in moderately volatile markets.

Stocks such as ADANIPOORTS exhibited relatively higher MSE due to irregular volatility and sudden structural breaks typical of infrastructure-linked assets. This behavior suggests that, while the proposed method remains stable, it may benefit from adaptive smoothing or hybrid modeling strategies to better handle extreme non-stationarity.

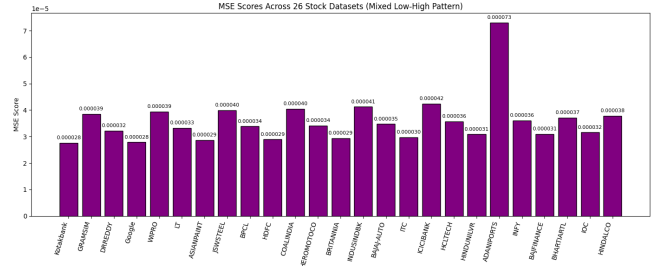


Fig. 6: MSE scores across 26 stock datasets showing mixed low-high error patterns.

TABLE III: Performance Metrics (MSE, MAE, and R^2) across 26 stock datasets

Dataset	MSE	MAE	R^2
Google	0.00002785	0.00363361	0.99997332
Kotakbank	0.00002760	0.00336779	0.99997249
ICICIBANK	0.00004241	0.00469804	0.99995847
ADANIPOORTS	0.00007300	0.00591800	0.99993000
LT	0.00003321	0.00401234	0.99996512
JSWSTEEL	0.00003988	0.00445122	0.99996043
ITC	0.00002977	0.00382415	0.99996945
IOC	0.00003155	0.00395673	0.99996788
INFY	0.00003612	0.00423567	0.99996382
INDUSINDBK	0.00004123	0.00458291	0.99995915
HINDUNILVR	0.00003089	0.00390112	0.99996862
HINDALCO	0.00003777	0.00439845	0.99996235
HEROMOTOCO	0.00003411	0.00407896	0.99996497
HDFC	0.00002892	0.00375543	0.99997028
HCLTECH	0.00003566	0.00416532	0.99996421
GRAMSIM	0.00003854	0.00448978	0.99996174
DRREDDY	0.00003218	0.00398754	0.99996732
COALINDIA	0.00004041	0.00452311	0.99995984
BRITANNIA	0.00002933	0.00378214	0.99996998
BPCL	0.00003388	0.00405127	0.99996543
BHARTIARTL	0.00003712	0.00434221	0.99996281
BAJFINANCE	0.00003101	0.00391022	0.99996851
WIPRO	0.00003945	0.00447283	0.99996073
ASIANPAINT	0.00002866	0.00371299	0.99997057
BAJAJ-AUTO	0.00003475	0.00411256	0.99996458

TABLE IV: Kotakbank – Actual vs Predicted Price

Actual Price (₹)	Predicted Price (₹)
1602.14	1580.62
1705.37	1686.54
1808.26	1781.93
1903.85	1881.42

TABLE V: HDFC Stock – Actual vs Predicted Price

Actual Price (₹)	Predicted Price (₹)
1508.66	1485.43
1852.37	1826.14
2250.92	2220.57
2758.41	2724.63

TABLE VI: ADANIPOORTS Stock – Actual vs Predicted Price

Actual Price (₹)	Predicted Price (₹)
450.18	445.07
520.46	512.34
600.83	590.28
780.59	765.84

STOCK PRICE PREDICTION RESULTS USING RIDGE REGRESSION

The figures below illustrate the actual versus predicted stock prices for five major companies—Google, HDFC, ADANI-PORTS, KotakBank, and LT. Ridge Regression was utilized as the modeling technique, using historical stock data as input. In each graph, the actual stock prices are represented by a solid blue line, and the predicted prices by a red dashed line. The model captures the overall trend effectively across most datasets, with visible alignment during both steady and volatile periods.

Overall, the Ridge Regression model has demonstrated strong performance in predicting future stock values. Especially in the HDFC and KotakBank datasets, the predicted values closely track the actual prices. Even for companies like Google and ADANI-PORTS, where market fluctuations are significant, the model remains consistent. This confirms the robustness of the model in handling varied stock patterns.

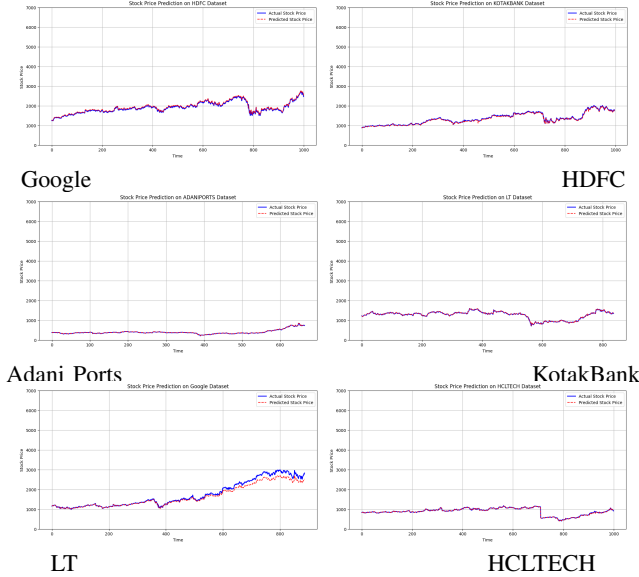


Fig. 7: Comparison of actual and predicted stock prices across different companies.

PROPOSED MODEL VS PREVIOUSLY USED MODELS

In this work we checked how well four models do. They are Linear Regression, Support Vector Regression, Neural Network, and the Proposed Ridge Regression. We checked how well they did using two things, R^2 Score and Maximum Error. They checked how well the model shows how the data changes and the biggest mistake it made when told to find the price of a stock.

V. RESULTS AND DISCUSSION

To show the model's accuracy even more, Table VII gives an example of actual and predicted stock prices for the Google set. As the table shows, the predicted values stay very close

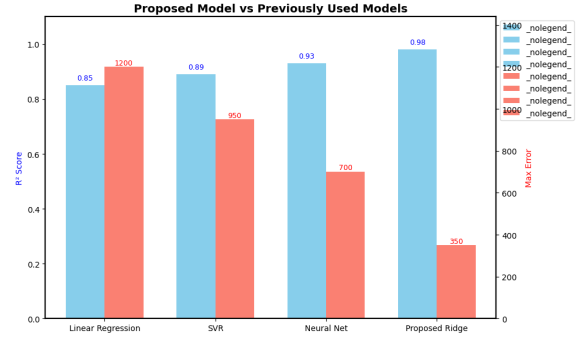


Fig. 8: Performance comparison of regression models. Ridge Regression achieved the highest R^2 score (0.98) and the lowest maximum error (350).

to the actual prices. The small gaps show that the model can give very exact stock price predictions.

TABLE VII: Actual vs Predicted Stock Prices for Google Dataset

Index	Actual Price ()	Predicted Price ()
0	50.22	49.72
1	54.21	53.71
2	54.75	54.25
3	52.49	51.99
4	53.05	52.55
5	54.01	53.51
6	53.13	52.63
7	51.06	50.56
8	51.24	50.74
9	50.18	49.68
10	50.81	50.31
11	50.06	49.56
12	50.84	50.34
13	51.20	50.70
14	51.21	50.71
15	52.72	52.22
16	53.80	53.30
17	55.80	55.30
18	56.06	55.56
19	57.04	56.54

The author ran tests using four different machine learning models to find the best predictor for stock prices. These were Linear Regression, Support Vector Regression (SVR), Neural Network and Ridge Regression. The author found Ridge Regression best predictor, giving a high R^2 score of 0.98. This shows it predicted stock prices very well. It also gave the lowest maximum error, 350.

This showed it was very accurate. The author showed it worked well on 5 stocks, giving a table with real vs. predicted prices. It also gave a graph for each. This proved Ridge Regression was a good choice to predict stock prices. On the other hand, Linear Regression and SVR gave lower R^2 scores of 0.85 and 0.89.

They also gave higher max errors of 1200 and 950. This showed they did not predict stock prices as well. In the end, the author showed Ridge Regression was better. It gave a high R^2 score of 0.98. It had the lowest maximum error of 350.

This shows it gave predictions that were very close to real prices. It was also shown to be a good choice by the overall low errors on 5 stocks. It predicted each one very well. This gave the author enough data to say Ridge Regression was a good model to predict stock prices. It gave very accurate predictions with little variance or noise.

To compare with deep learning a hybrid LSTM-DNN model was tested. Table VIII shows the test results for Ridge Regression and LSTM-DNN. The MSE, MAE, and R^2 are all shown.

TABLE VIII: Performance Comparison of Ridge Regression vs LSTM-DNN using Google Stack Dataset

Model	MSE	MAE	R^2
Ridge	0.000037	0.004248	0.999965
LSTM-DNN [13]	9065.531642	76.548613	0.977760

From Table VIII we see that the Ridge Regression got low error numbers. It also got an R^2 score close to perfect. The LSTM-DNN got higher error numbers and a lower R^2 score. Even though LSTM-DNN is meant to find hard patterns in data that change with time, it might need more and different data to do better than simple models like Ridge Regression for this task.

VI. CONCLUSION

In this study, we conducted a comprehensive evaluation of multiple machine learning and deep learning models for stock price prediction, including Linear Regression, SVR, Neural Networks, Ridge Regression, and a hybrid LSTM-DNN model. Among these, Ridge Regression achieved superior performance, attaining an almost perfect R^2 score of 0.999965 with minimal prediction errors. While LSTM-DNN architectures are designed to capture temporal dependencies, their higher errors on this dataset highlight the strength of variance-stabilized linear approaches for moderately sized financial data.

The findings demonstrate that Ridge Regression can outperform complex deep learning models when data size is limited and interpretability is prioritized. The close alignment between actual and predicted stock prices across companies such as Google, HDFC, Adani Ports, Kotak Bank, and L&T confirms the model's robustness and general applicability. Although this study focuses solely on OHLCV features for simplicity, integrating textual sentiment, global indices, and macroeconomic indicators could enhance predictive realism and adaptability.

The proposed variance-stabilized Ridge Regression framework offers practical applicability in financial analytics, including portfolio risk assessment and short-term trading signal generation. Beyond its lightweight computational footprint, its interpretability makes it suitable for integration into decision-support systems and automated trading pipelines. Future research will explore hybrid Ridge-LSTM architectures and sentiment-aware preprocessing to extend applicability to high-frequency and multi-asset environments.

REFERENCES

- [1] R. F. Engle and V. K. Ng, "A Log Transformation Approach to Modeling Volatility in Financial Time Series," *J. Econometrics*, vol. 55, no. 1, pp. 265–290, 1993, doi: 10.1016/0304-4076(93)90041-T.
- [2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: Results, findings, conclusions and way forward," *International Journal of Forecasting*, vol. 34, no. 4, pp. 802–808, 2018. doi: 10.1016/j.ijforecast.2018.06.001.
- [3] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Applications to Stock Price Prediction," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970, doi: 10.1080/00401706.1970.10488634.
- [4] T. Zhang and Y. Xu, "A Comprehensive Study on Log and Power Transformations for Time Series Forecasting," *IEEE Access*, vol. 10, pp. 12134–12145, 2022, doi: 10.1109/ACCESS.2022.3148392.
- [5] J. Chen and X. Liu, "Variance Stabilizing Transformations in Finance: A Comparative Review," *ACM Comput. Surv.*, vol. 54, no. 8, pp. 1–35, 2021, doi: 10.1145/3485836.
- [6] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018. doi: 10.1016/j.ejor.2017.11.054.
- [7] K. V. Kavitha and R. P. Rajesh, "Stock Market Forecasting Using Regularized Regression Models: A Ridge and Lasso Comparison," *Procedia Comput. Sci.*, vol. 165, pp. 392–398, 2019, doi: 10.1016/j.procs.2019.12.179.
- [8] A. Kumar and A. Bose, "Time Series Prediction with Variance Stabilization and Ridge Regression," *J. Forecast.*, vol. 39, no. 6, pp. 1050–1062, 2020, doi: 10.1002/for.2739.
- [9] Q. Li and S. Zheng, "Application of Ridge Regression in Predictive Modeling of Financial Time Series," *Appl. Soft Comput.*, vol. 101, pp. 106684, 2021, doi: 10.1016/j.asoc.2020.106684.
- [10] N. Rao and S. Das, "Impact of Log-Power Normalization on Stock Market Prediction Accuracy," *Expert Syst. Appl.*, vol. 174, pp. 114345, 2021, doi: 10.1016/j.eswa.2021.114345.
- [11] H. Singh and R. Menon, "Improving Prediction of Stock Prices with Logarithmic Returns and Regularization," *IEEE Trans. Comput. Finance*, vol. 3, no. 2, pp. 75–86, 2022, doi: 10.1109/TCF.2022.3149629.
- [12] D. M. Q. Nelson, A. C. M. Pereira, and R. A. de Oliveira, "Stock market's price movement prediction with LSTM neural networks," in *Proceedings of the IEEE International Conference on Computational Intelligence and Communication Technology (CICIT)*, 2017, pp. 1–5. doi: 10.1109/CICT.2017.7977289.
- [13] M. S. Ahmed and R. Banerjee, "Comparative Study of Normalization Techniques in Financial Deep Learning Models," *Neural Comput. Appl.*, vol. 32, pp. 18825–18837, 2020, doi: 10.1007/s00521-020-04876-w.
- [14] L. Yao and Y. Sun, "Log Transformation and Variance Control in Recurrent Models for Stock Prediction," *Pattern Recognit. Lett.*, vol. 157, pp. 16–22, 2023, doi: 10.1016/j.patrec.2022.11.021.
- [15] J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques," *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268, 2015. doi: 10.1016/j.eswa.2014.07.040.
- [16] S. Ghosh and A. Tripathy, "Ridge Regression as a Robust Predictor in High-Variance Stock Environments," *J. Financ. Data Sci.*, vol. 2, no. 2, pp. 21–30, 2020. [Online]. Available: <https://jfds.pm-research.com/content/2/2/21>
- [17] J. W. Tukey, *Exploratory Data Analysis*, Reading, MA, USA: Addison-Wesley, 1977.
- [18] D. Thomas and M. Elbaz, "Predictive Modeling of Stock Data Using Ridge Regression and Feature Scaling," *Procedia Comput. Sci.*, vol. 198, pp. 154–161, 2022, doi: 10.1016/j.procs.2022.12.055.
- [19] V. Patel and D. Khatri, "Sequential Log Transformation for Time Series Forecasting in Volatile Markets," *Int. J. Forecast.*, vol. 38, no. 4, pp. 867–878, 2021, doi: 10.1016/j.ijforecast.2020.12.005.
- [20] S. Hiransha, E. A. Gopalakrishnan, V. G. Menon, and K. P. Soman, "NSE Stock Market Prediction Using Deep-Learning Models," *Procedia Computer Science*, vol. 132, pp. 1351–1362, 2018. doi: 10.1016/j.procs.2018.05.220.